

UNIVERSIDAD INTERNACIONAL DEL ECUADOR

Ingeniería en Sistemas de la Información



Informe Técnico

Nombre

Francis Ariel Esparza Melo

Asignatura

Fundamentos De La Seguridad De Los Sistemas Informáticos Y De La Información

Docente

Dario Sebastián Cabezas Erazo

Fecha

13/05/2026

Índice

Introducción.....	3
Marco Teórico.....	3
Desarrollo y Evidencias	5
Análisis y Reflexión	10
Referencias	11

Introducción

Para empezar, tenemos que entender cual es el contexto del problema, hoy en día las aplicaciones web guardan mucha información de cada usuario una de ella son las contraseñas y estas son las mas vulnerables ya que si se almacenan mal puede haber una brecha lo cual puede dar como consecuencias la exposición de todos los usuarios.

Porque es importante la seguridad en el almacenamiento, muchos sistemas guardan información como lo son las contraseñas en texto plano o con algoritmos como MD5 los cuales pueden abrir brechas como lo que paso con LinkedIn en 2012 expuso las contraseñas hasheadas de aproximadamente 117 millones de cuentas, las cuales fueron descifradas y vendidas en línea debido al uso de SHA-1 sin salt (Gune, 2017).

El objetivo del proyecto es desarrollar una API de autenticación segura utilizando FastAPI y SQLAlchemy, aplicando técnicas criptográficas modernas para proteger las credenciales de los usuarios, específicamente mediante la implementación de hashing con bcrypt, salting automático por usuario y peppering mediante variables de entorno del servidor.

Marco Teórico

El hashing es un proceso criptográfico que transforma cualquier dato de entrada en una cadena de longitud fija, conocida como hash o resumen. Este proceso es unidireccional, lo que significa que no es posible recuperar el dato original a partir del hash generado. En el contexto de la seguridad informática, el hashing se utiliza para almacenar contraseñas de forma segura, ya que, en lugar de guardar la contraseña en texto plano, el sistema guarda únicamente su hash. Cuando el usuario inicia sesión, el sistema repite el proceso con la contraseña ingresada y compara los resultados (OWASP, s.f.).

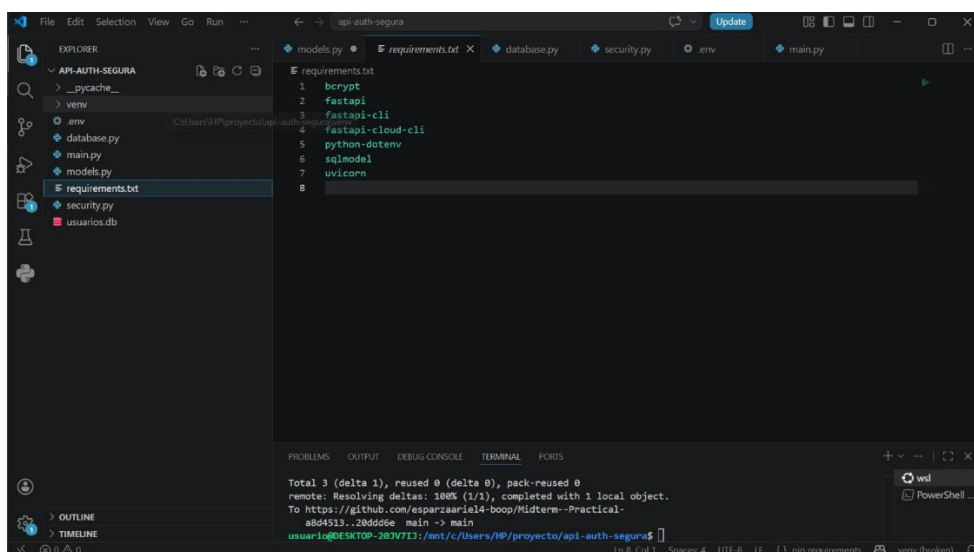
Sin embargo, no todos los algoritmos son adecuados para proteger contraseñas como lo son MD5 o SHA-256 ya que estos representan un riesgo crítico en el almacenamiento de contraseñas. Aunque fueron diseñados para verificar con velocidad esto mismo se convierte en una vulnerabilidad cuando se aplican a credenciales de usuario, ya que un atacante puede probar miles de millones de combinaciones por segundo. Un ejemplo real de las consecuencias de estas malas prácticas fue la filtración de LinkedIn en 2012, donde aproximadamente 117 millones de contraseñas hashadas con SHA-1 sin salt fueron descifradas y vendidas en línea (Gune, 2017).

Bcrypt es un algoritmo de hashing diseñado específicamente para el almacenamiento seguro de contraseñas. A diferencia de MD5 o SHA-256, bcrypt incorpora un factor de costo configurable que determina el número de rondas de procesamiento aplicadas al hash. Cada incremento en este factor duplica el tiempo de cálculo, lo que hace que los ataques de fuerza bruta sean computacionalmente inviables. Para un usuario que inicia sesión una sola vez, este tiempo es imperceptible, pero para un atacante que intenta millones de combinaciones, representa una barrera prácticamente insuperable (Mazières, 1999).

Para mejorar la seguridad en el almacenamiento de contraseñas, se utilizan técnicas como lo son salt y pepper. El salt es un valor aleatorio único generado por cada usuario que se agrega a la contraseña antes del hashing, evitando que dos usuarios con la misma contraseña produzcan hashes idénticos. El pepper, por su parte, es una cadena secreta definida a nivel de servidor que se concatena a la contraseña antes del proceso de hashing, pero a diferencia del salt, nunca se almacena en la base de datos sino en las variables de entorno del servidor. Lo que quiere decir que si un atacante entra a la base de datos no puede reducir el proceso de verificación por que desconoce del pepper (Grassi, 2017) (OWASP, s.f.).

Desarrollo y Evidencias

El proyecto fue desarrollado en un entorno WSL mediante el subsistema de Windows, utilizando Python 3.12 con un entorno virtual para poder instalar las dependencias del proyecto. Para gestionar el proyecto y la base de datos se instalaron las siguientes librerías FastAPI como framework principal para la construcción de la API, Uvicorn como servidor ASGI para ejecutar la aplicación, SQLAlchemy para la interacción con la base de datos SQLite, bcrypt para el proceso de hashing de contraseñas, y python-dotenv para la gestión del pepper mediante variables de entorno. La base de datos SQLite se genera automáticamente al levantar el servidor o la aplicación, sin necesidad de configurar algo adicional.



The screenshot shows a Visual Studio Code editor window with the following components:

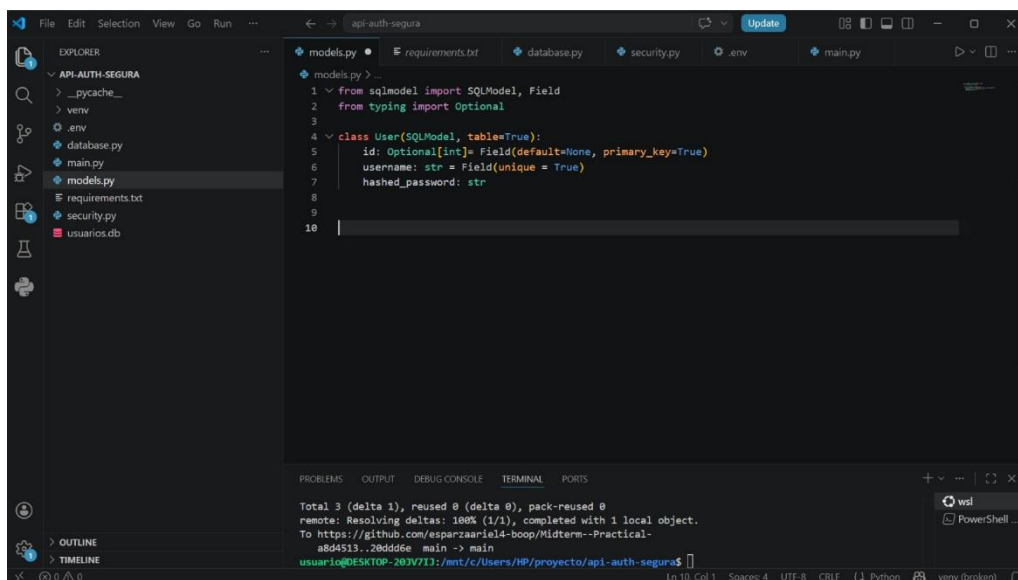
- EXPLORER:** A file tree on the left showing the project structure: `API-AUTH-SEGURA` (folder), `__pycache__` (folder), `venv` (folder), `env` (file), `database.py` (file), `main.py` (file), `models.py` (file), `requirements.txt` (file), `security.py` (file), and `usuarios.db` (file).
- EDITOR:** The `requirements.txt` file is open, showing the following dependencies:

```
1 bcrypt
2 fastapi
3 fastapi-cli
4 fastapi-cloud-cli
5 python-dotenv
6 sqlalchemy
7 uvicorn
```
- TERMINAL:** The terminal at the bottom shows the output of a `pip install` command:

```
Total 3 (delta 1), reused 0 (delta 0), pack-reused 0
remote: Resolving deltas: 100% (1/1), completed with 1 local object.
To https://github.com/esparzaarield-boop/Midterm-Practical-
ad64513..28d6d06 main -> main
usuario@DESKTOP-28V7T3: /mnt/c/Users/HP/proyecto/api-auth-segura$
```

Para el almacenamiento de usuarios se usó un modelo de datos utilizando SQLAlchemy, el cual permite mapear clases de Python directamente a tablas de base de datos. El modelo contiene únicamente lo necesario para la autenticación: un identificador único que se genera automáticamente, el nombre del usuario con una restricción única para evitar registros duplicados y el hash de la contraseña. Con este diseño se busca lo minimalista lo cual reduce la

cantidad de información expuesta en caso de una brecha de seguridad ya que en ningún momento se expone la contraseña en texto plano.



```

1 from sqlalchemy import SQLModel, Field
2 from typing import Optional
3
4 class User(SQLModel, table=True):
5     id: Optional[int] = Field(default=None, primary_key=True)
6     username: str = Field(unique=True)
7     hashed_password: str
8
9
10

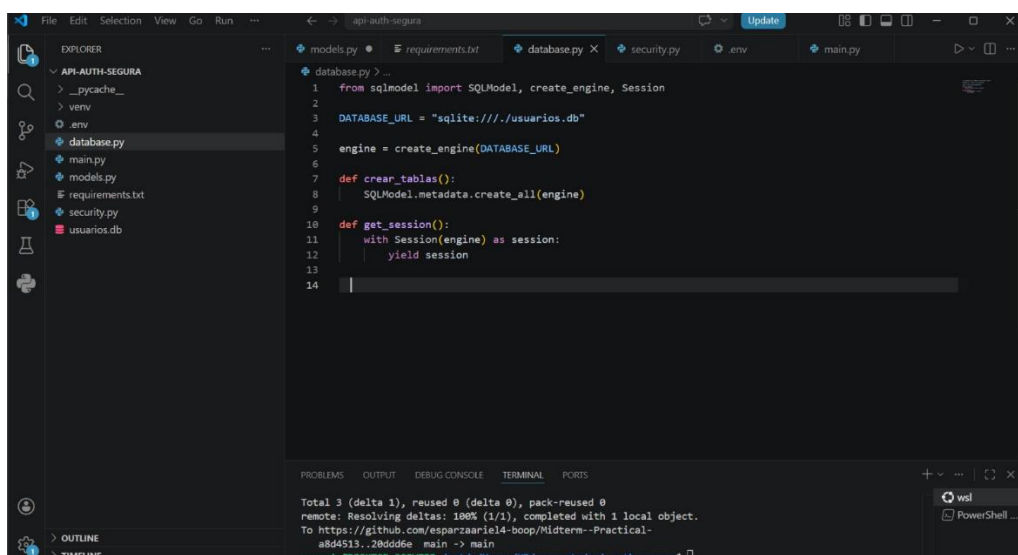
```

Terminal output:

```

Total 3 (delta 1), reused 0 (delta 0), pack-reused 0
remote: Resolving deltas: 100% (1/1), completed with 1 local object.
To https://github.com/esparzaariel4-boop/Midterm-Practical-
a8d4513..28dd6e main -> main
usuario@DESKTOP-28JVT13: /mnt/c/Users/HP/proyecto/api-auth-segura$

```



```

1 from sqlalchemy import SQLModel, create_engine, Session
2
3 DATABASE_URL = "sqlite:///usuarios.db"
4
5 engine = create_engine(DATABASE_URL)
6
7 def crear_tablas():
8     SQLModel.metadata.create_all(engine)
9
10 def get_session():
11     with Session(engine) as session:
12         yield session
13
14

```

Terminal output:

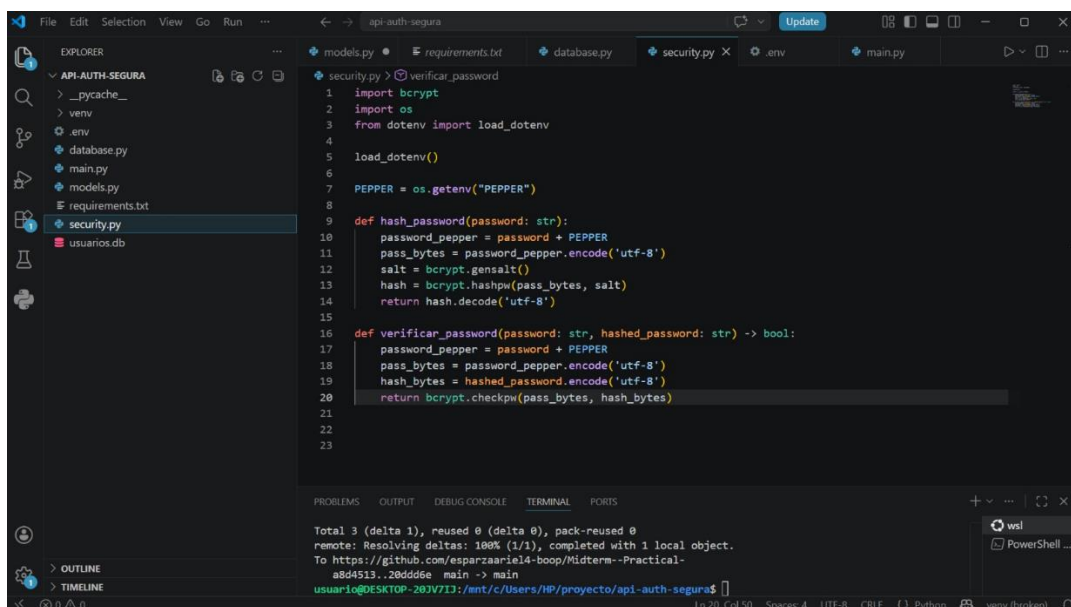
```

Total 3 (delta 1), reused 0 (delta 0), pack-reused 0
remote: Resolving deltas: 100% (1/1), completed with 1 local object.
To https://github.com/esparzaariel4-boop/Midterm-Practical-
a8d4513..28dd6e main -> main
usuario@DESKTOP-28JVT13: /mnt/c/Users/HP/proyecto/api-auth-segura$

```

La seguridad de contraseñas se implementó en un archivo dedicado utilizando la librería bcrypt. El proceso inicia recuperando el pepper desde las variables de entorno del servidor mediante python-dotenv, garantizando que este valor secreto nunca esté visible en el código fuente ni en la base de datos. Al momento de registrar un usuario, la contraseña recibida en texto plano se concatena con el pepper, se convierte a bytes y se pasa a bcrypt, que genera

automáticamente un salt único y produce el hash final. Para la verificación en el login, se repite exactamente el mismo proceso y se compara el resultado con el hash almacenado en la base de datos mediante la función `checkpw` de `bcrypt`.

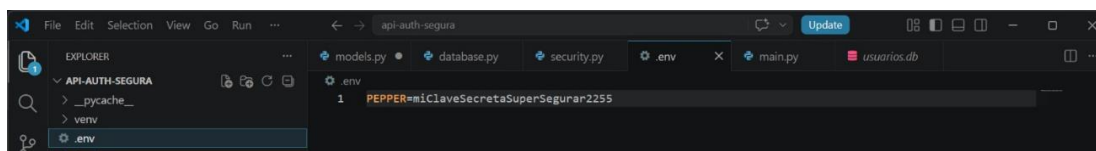


```

1 import bcrypt
2 import os
3 from dotenv import load_dotenv
4
5 load_dotenv()
6
7 PEPPER = os.getenv("PEPPER")
8
9 def hash_password(password: str):
10     password_pepper = password + PEPPER
11     pass_bytes = password_pepper.encode('utf-8')
12     salt = bcrypt.gensalt()
13     hash = bcrypt.hashpw(pass_bytes, salt)
14     return hash.decode('utf-8')
15
16 def verificar_password(password: str, hashed_password: str) -> bool:
17     password_pepper = password + PEPPER
18     pass_bytes = password_pepper.encode('utf-8')
19     hash_bytes = hashed_password.encode('utf-8')
20     return bcrypt.checkpw(pass_bytes, hash_bytes)
21
22
23

```

The terminal shows the command `main -> main` and the output: `Total 3 (delta 1), reused 0 (delta 0), pack-reused 0`.



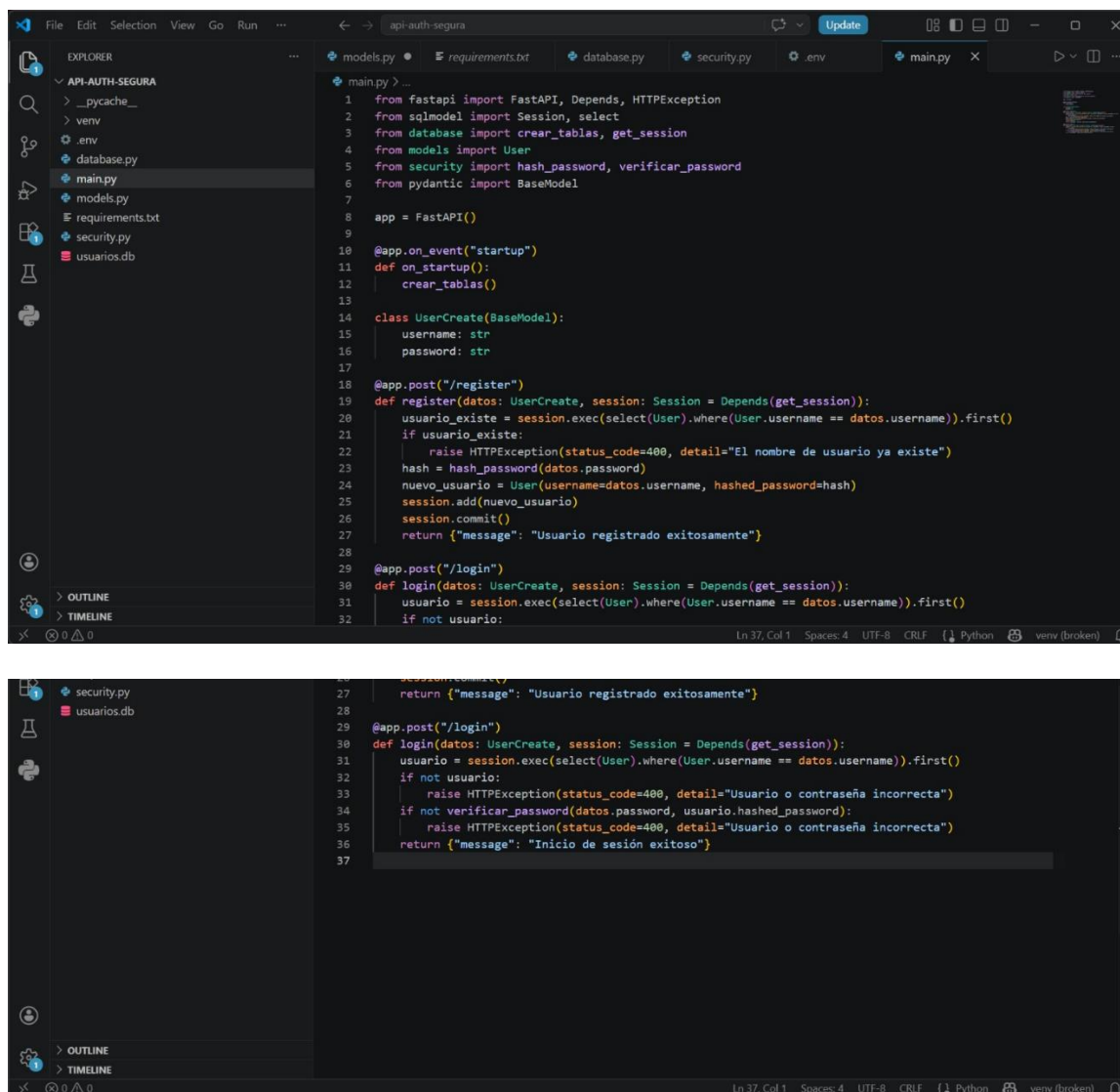
```

1 PEPPER=miClaveSecretaSuperSegura2255

```

Se desarrollaron dos endpoints REST utilizando FastAPI. El endpoint `POST /register` recibe el nombre de usuario y la contraseña en texto plano, verifica que el nombre de usuario no exista previamente en la base de datos, aplica el proceso de hashing seguro con pepper y salt, y almacena el nuevo usuario con su hash correspondiente. En caso de que el usuario ya exista, el sistema retorna un error 400 con un mensaje informando el problema. El endpoint `POST /login` busca al usuario en la base de datos por su nombre de usuario, y si existe, verifica la contraseña ingresada contra el hash almacenado utilizando la función de comparación de `bcrypt`. Si lo que ingreso el usuario son incorrectas, el sistema retorna el mismo mensaje de error tanto para

usuario inexistente como para contraseña incorrecta, evitando revelar información al atacante sobre cuál de los dos datos falló.



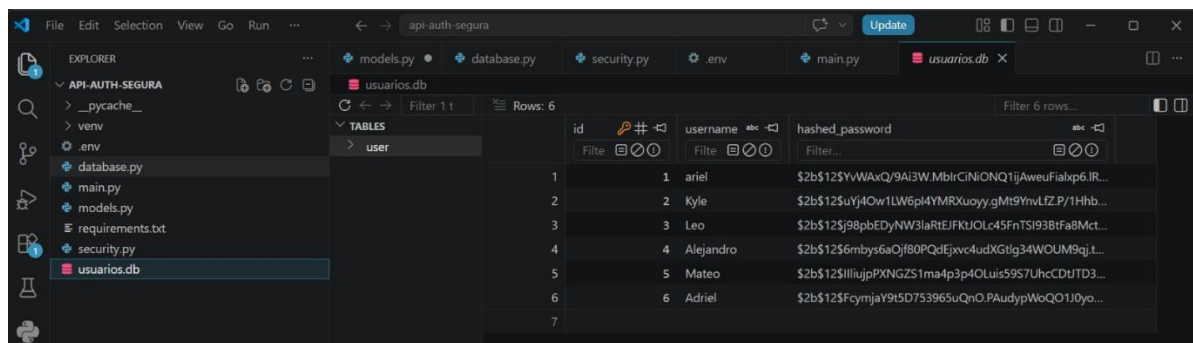
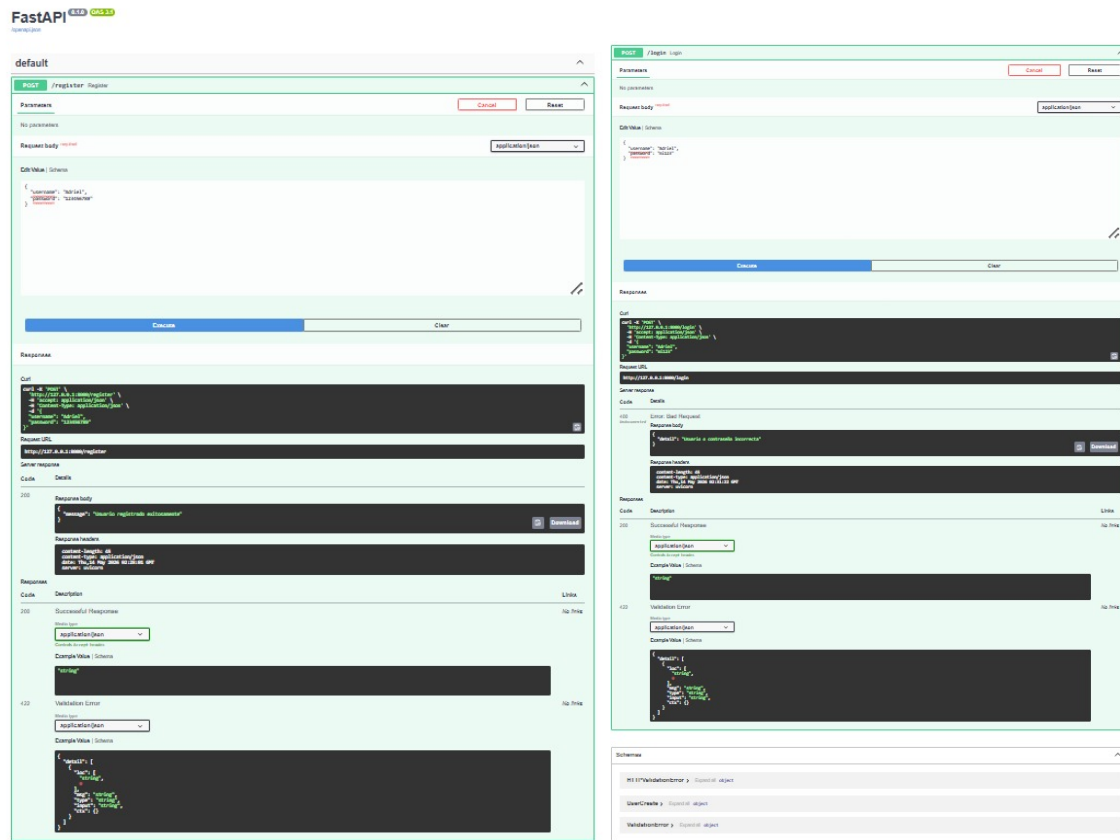
```

1  from fastapi import FastAPI, Depends, HTTPException
2  from sqlmodel import Session, select
3  from database import crear_tablas, get_session
4  from models import User
5  from security import hash_password, verificar_password
6  from pydantic import BaseModel
7
8  app = FastAPI()
9
10 @app.on_event("startup")
11 def on_startup():
12     crear_tablas()
13
14 class UserCreate(BaseModel):
15     username: str
16     password: str
17
18 @app.post("/register")
19 def register(datos: UserCreate, session: Session = Depends(get_session)):
20     usuario_existe = session.exec(select(User).where(User.username == datos.username)).first()
21     if usuario_existe:
22         raise HTTPException(status_code=400, detail="El nombre de usuario ya existe")
23     hash = hash_password(datos.password)
24     nuevo_usuario = User(username=datos.username, hashed_password=hash)
25     session.add(nuevo_usuario)
26     session.commit()
27     return {"message": "Usuario registrado exitosamente"}
28
29 @app.post("/login")
30 def login(datos: UserCreate, session: Session = Depends(get_session)):
31     usuario = session.exec(select(User).where(User.username == datos.username)).first()
32     if not usuario:
33         raise HTTPException(status_code=400, detail="Usuario o contraseña incorrecta")
34     if not verificar_password(datos.password, usuario.hashed_password):
35         raise HTTPException(status_code=400, detail="Usuario o contraseña incorrecta")
36     return {"message": "Inicio de sesión exitoso"}
37

```

Para verificar el correcto funcionamiento del sistema de seguridad, se usó la base de datos SQLite utilizando la extensión SQLite Viewer en Visual Studio Code. Se registraron varios usuarios, incluyendo algunos con contraseñas idénticas, con el objetivo de comprobar que el salt funciona correctamente. Los resultados confirmaron que las contraseñas almacenadas son completamente ilegibles, todos los hashes comienzan con 2b\$12 indicando el uso de bcrypt con

12 rondas de costo, y usuarios con la misma contraseña presentan hashes completamente diferentes gracias al salt único generado por cada registro.



Para una mejor visualización de todo el proyecto lo puede encontrar en el repositorio creado para el mismo con el siguiente enlace: <https://github.com/esparzaariel4-boop/Midterm--Practical->

Análisis y Reflexión

Si un atacante logra descargar el archivo usuarios.db obtiene únicamente los hashes de las contraseñas. Ya que el pepper se almacena exclusivamente en las variables de entorno del servidor y no en la base de datos, el atacante no puede reproducir el proceso de hashing correctamente. Aunque conozca el algoritmo utilizado y tenga los hashes, cualquier intento de verificar contraseñas candidatas producirá resultados incorrectos porque le falta el componente secreto que se concatenó antes del hashing. Esto convierte el ataque en prácticamente imposible de realizar, ya que el atacante necesitaría comprometer simultáneamente tanto la base de datos como el servidor para obtener el pepper.

El costo en bcrypt es un parámetro configurable que determina cuántas rondas de procesamiento se aplican para generar el hash. En este proyecto se utilizó el valor por defecto de 12 rondas, se representa como $2b\$12$ al inicio de cada hash almacenado. Cada incremento de una unidad en este factor duplica el tiempo de cálculo, lo que significa que pasar de costo 11 a costo 12 dobla el tiempo necesario para generar un hash. Para un usuario este tiempo es imperceptible, pero para un atacante que intenta millones de contraseñas por segundo, este costo convierte el ataque de fuerza bruta en una operación prácticamente imposible de ejecutar con el método de seguridad realizado en este proyecto.

Referencias

Grassi, P. A. (2017). *Digital identity guidelines*. NIST.

Gune, A. (2017). *The Cryptographic Implications of the LinkedIn*. Corvallis: Department of Electrical Engineering and Computer Science.

Mazières, N. P. (1999). *A Future-Adaptable Password Scheme*. Monterey, California: USENIX.

OWASP. (s.f.). *OWASP*. Obtenido de Password Storage Cheat Sheet:
https://cheatsheetseries.owasp.org/cheatsheets/Password_Storage_Cheat_Sheet.html